

QA Process & Best Practices

Shift-Left • Agile QA • Defect Management • Metrics • Team Standards

Brian Scott McCarthy | Senior QA Lead | QA Best Practice Series — BP-06

1. QA Philosophy — Quality is a Team Sport

Quality is not the QA team's sole responsibility. QA's role is to **protect the product and the customer** by: (1) preventing defects through early involvement, (2) detecting defects before they reach production, and (3) providing the data teams need to make confident release decisions.

2. Shift-Left Testing

Shift-left means involving QA earlier in the process — before code is written. Defects caught in requirements cost 10–100x less to fix than defects caught in production.

Activity	Traditional (Right)	Shift-Left
QA involvement begins	After development is complete	At story refinement / sprint planning
Test case authoring	After feature is built	During or before development (acceptance criteria → test cases)
Defect discovery	QA test phase	During development (unit tests, code review, QA pairing)
API testing	After UI is complete	As soon as API contract (Swagger spec) is defined
Automation	After full feature delivery	Automation candidate identified at story creation

Practical shift-left actions for QA engineers:

- Attend backlog refinement — flag missing acceptance criteria, ambiguous requirements, and untestable stories before the sprint begins.
- Review Swagger/OpenAPI specs before the API is built — catch design flaws early.
- Author test cases from acceptance criteria during the sprint — before the feature is complete — so execution can begin immediately on delivery.
- Request a dev demo of each feature before formal QA — spot integration issues and missing states early.
- Raise a defect as soon as a bug is found — do not batch bugs for end-of-sprint reporting.

3. Agile QA Rituals — What QA Does in Each Ceremony

Ceremony	QA Participation
Sprint Planning	Confirm all stories have testable acceptance criteria; estimate QA effort; flag dependencies or environment risks.
Backlog Refinement	Review upcoming stories for testability; propose acceptance criteria; identify automation candidates; flag unclear requirements.
Daily Standup	Report: what was tested yesterday, what is being tested today, any blockers (environment issues, missing test data, build failures).
Sprint Review	Present QA metrics: test cases executed, pass rate, defects found/resolved, automation coverage change vs last sprint.

Ceremony	QA Participation
Sprint Retrospective	Raise process issues affecting QA: late requirements, environment instability, test data problems, automation debt.
Go/No-Go Meeting	Present test summary report; state QA recommendation (Go / Conditional Go / No-Go) with supporting metrics.

4. Defect Life Cycle

State	Who Owns It	Next Action
New	QA Engineer	Log in ADO with full details; assign to Dev Lead
Active	Dev Lead / Developer	Investigate and fix; update with root cause notes
Resolved	Developer	Mark Resolved; build number where fix is included; notify QA
Verify	QA Engineer	Re-execute failing test case in correct build; verify fix
Closed	QA Engineer	If fix verified → Close; add verification note and build number
Won't Fix	Product Owner	Risk accepted; documented in release notes; QA closes with note
Duplicate	QA Lead	Link to original bug; close duplicate; original remains active
Cannot Reproduce	QA Engineer + Dev	Add additional reproduction info; if still not repro → close with note

5. Test Data Management

Principle	Practice
Never use production data in QA	Use anonymised or synthetic data; production data in QA environments creates privacy and compliance risk
Seed data is documented	All required test accounts, products, and records are documented in the ADO wiki and version-controlled in Git
Data reset procedure exists	A documented script or process restores the QA DB to a known state; run before full regression cycles
Data is independent per test	Each automated test creates its own data or uses dedicated test accounts — never shares mutable state
Sensitive data is masked	Passwords, card numbers, and PII in test data use recognisable patterns (4242 4242 4242 4242; testuser@qa.com)
Environments have dedicated data	QA, Staging, and UAT each have separate datasets — a test in Staging never pollutes QA data

6. QA Metrics — What to Measure & Why

Metric	Formula / Source	Target	Why It Matters
Test execution rate	$\text{Cases executed} / \text{cases planned} \times 100$	100% by sprint end	Confirms QA scope was fully covered

Metric	Formula / Source	Target	Why It Matters
Pass rate	Cases passed / cases executed × 100	> 90% at sprint end	Indicates stability of build under test
Defect escape rate	Production bugs / sprint total bugs × 100	< 5% (target 0 for P1)	Measures QA effectiveness at catching bugs before release
P1 defect escape	P1 bugs reaching production	0	Single most important QA quality metric
Defects per sprint	Total bugs logged per sprint	Trending down	Measures improving development quality over time
Automation pass rate	Automated tests passed / total run × 100	> 95%	Indicates automation suite health and reliability
Automation coverage	Automated test cases / total test cases × 100	Growing; > 60% for P1-P2	Shows regression protection level
Regression cycle time	Hours to complete full regression	Decreasing sprint-over-sprint	Demonstrates automation investment paying off
Flaky test rate	Flaky test runs / total automated runs × 100	< 2%	High flakiness erodes trust in automation

7. Environment Management

Environment	Purpose	QA Actions
Development (Dev)	Developer local/branch environment	QA may assist with early API contract testing; not a formal QA environment
QA	Primary QA testing environment	All functional test execution; smoke testing of every build; defect logging
Staging	Production-mirror environment	Full regression; UAT; performance testing; final pre-release validation
Production	Live environment	Post-deployment smoke test only (critical paths); monitoring alerts

Environment Rule: QA must not be blocked by environment issues for more than 2 hours without escalation. Log environment issues in ADO immediately; notify DevOps via Slack; escalate to QA Lead if unresolved in 2 hours.

8. API-First Testing

Always test the API layer before testing the UI layer. API testing is faster, more reliable, and finds defects at the root cause — not at the symptom level.

- When a new API endpoint is added, test it with Postman before any UI that calls it is built.
- Use REST Assured or PyTest for automated API regression — these run in CI on every PR.
- Validate response schema against the Swagger/OpenAPI spec on every API test run.
- Test error responses (400, 401, 403, 404, 422, 500) as rigorously as success responses.
- Include API tests in the smoke suite — a passing UI can mask a broken API response.
- Log API defects with the endpoint, method, request body, response body, and HTTP status code.

9. Communication Standards

Situation	Action	Channel
P1 defect found	Log in ADO immediately; notify Dev Lead and Product Owner	ADO Bug + Slack #qa-alerts
Build deployed to QA env	Run smoke suite; post pass/fail result within 30 min	Slack #deployments
Environment is down	Log in ADO; notify DevOps; update team	Slack #qa-channel + @DevOps
Blocked on missing test data	Log blocker in ADO standup; request from Dev or DBA	ADO standup + Slack DM to Dev
Sprint test summary ready	Share ADO report link in sprint review + team channel	ADO Test Plans + Slack #sprint-review
Go/No-Go recommendation	Present in Go/No-Go meeting; email summary to stakeholders	Meeting + email

10. QA Engineer Daily Checklist

Use this checklist to structure each working day:

Time	Activity
Start of day	Check overnight CI pipeline results in ADO — triage any failures
Start of day	Check if any new builds were deployed to QA env overnight — run smoke suite if yes
Morning	Execute assigned test cases for the sprint in Azure DevOps Test Plans
Ongoing	Log all defects in ADO immediately on discovery — do not batch
Ongoing	Verify resolved defects as soon as dev marks them fixed — do not let resolved bugs sit in queue
End of day	Update ADO test case outcomes — no untouched test cases at day end without a comment
End of day	Post a brief standup update in Slack: tested X, found Y bugs, blocked on Z
End of sprint	Complete test summary report in ADO; prepare Go/No-Go data for sprint review